

Kafka Production Patterns

Cheat Sheet Series for All Levels



**Kafka works in dev.
Production makes it honest.**



The Problem



Partition growth increases cost and operational complexity



Consumer lag causes timeouts and stale processing



Retries and poison messages can create failure loops



Ordering, replay and duplicates are hard in real systems



Without observability, incidents take too long to diagnose

What production teams actually optimize



Reliability



Ordering



Replay



Observability



Cost



Pattern mindset: design for failure, reprocessing and scale from day one.

Kafka Production Patterns

Beginner Fundamentals



Think of Kafka as a durable event log for moving business events.



1. Event

a fact that happened, like OrderCreated



2. Producer

publishes events to Kafka



3. Topic

a named stream of related events



4. Partition

a shard that enables scale and ordering per key



5. Broker

a Kafka server that stores and serves data



6. Consumer

reads events and processes them



7. Consumer Group

multiple consumers sharing work



8. Offset

the position of a consumer in a partition

Why teams use Kafka



Decoupling



Scalability



Replay



Durability



Integration



Rule of thumb: choose a stable message key when ordering matters.

Kafka Production Patterns

Cheat Sheet Series for All Levels

Core Terminology

1



Replication

copies of partition data for fault tolerance

2



Leader / Follower

leader serves reads/writes; followers replicate

3



ISR

in-sync replicas eligible for safe writes

4



acks=all

producer waits for strongest acknowledgement

5



Idempotent Producer

avoids duplicates caused by retries

6



Dead Letter Topic (DLT)

stores failed messages for analysis

7



Retry Topic

delayed reprocessing before final failure handling

8



Retention

how long Kafka keeps data

9



Compaction

keeps the latest value per key

10



Rebalance

consumers redistribute partitions when group membership changes



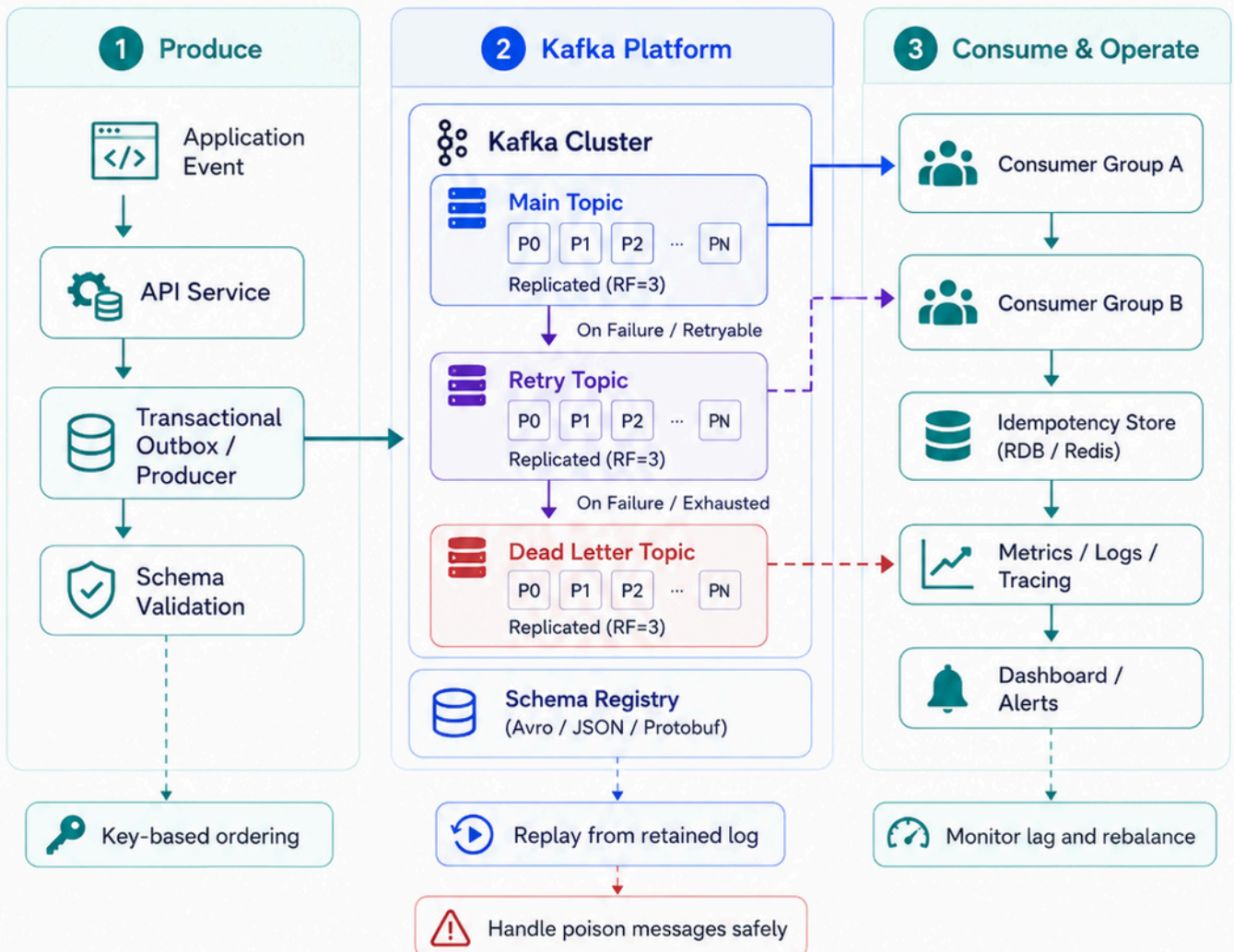
Exactly-once is a system property, not a single toggle.



Know these terms before tuning throughput or debugging incidents.

Kafka Production Patterns

Architecture Diagram



Production pattern flow: publish safely, process reliably, recover predictably.

Kafka Production Patterns

Cheat Sheet Series for All Levels



Spring Boot Implementation



Producer essentials

```
spring.kafka.producer.acks=all
spring.kafka.producer.properties.
enable.idempotence=true
spring.kafka.producer.compression-
type=zstd
spring.kafka.producer.retries=10
```

Send with key (ordering)

```
kafkaTemplate.send("orders.v1",
orderId, event);
```



Consumer essentials

```
@KafkaListener(
    topics = "orders.v1",
    groupId = "billing-service")
public void handle(OrderCreated
event) { ... }
```



Error handling



retry topic



dead letter
topic



manual ack
when needed



idempotent
processing



Implementation tips

1

Choose a key
that matches
your ordering
boundary

2

Version your
event
contracts

3

Keep payloads
focused and
explicit

4

Test replay
and failure
handling early



Pattern mindset: design for failure, reprocessing and scale from day one.

Kafka Production Patterns

Intermediate Patterns

1



Transactional Outbox



Solves: dual-write problems between DB and Kafka



Trade-off: extra table/process

2



Idempotent Consumer



Solves: handles duplicate deliveries safely



Trade-off: extra state or dedupe store

3



Retry Topics + DLT



Solves: isolates transient failures from poison messages



Trade-off: more operational flow

4



Key-Based Ordering



Solves: preserves order per entity like orderId or accountId



Trade-off: hot partitions if keys are skewed

5



Log Compaction



Solves: keeps the latest state per key for stateful consumers



Trade-off: not for audit-style full history alone



Production patterns reduce chaos by making failure paths explicit.



Use patterns intentionally—each one buys reliability with a cost.



Kafka

Production Patterns

Cheat Sheet Series for All Levels



Production Configuration



Topic & Broker



replication.factor ≥ 3



min.insync.replicas = 2



retention.ms based on replay needs



cleanup.policy = delete or compact



Producer



acks = all



enable.idempotence = true



compression.type = zstd



batch.size and linger.ms tuned for throughput



Consumer



max.poll.records tuned to processing capacity



max.poll.interval.ms sized to worst-case processing



isolation.level = read_committed when using transactions



static membership reduces rebalance churn



Observability



track consumer lag



watch rebalance count



monitor throughput, errors, retry volume



alert on DLT growth and broker health



Tune with benchmarks, not guesswork.

Kafka Production Patterns

Cheat Sheet Series for All Levels



Failure Scenarios

Failure	Symptom	Response
 1. Consumer lag spike	backlog grows, SLAs miss	scale consumers, inspect hot partitions, tune processing
 2. Poison message	same record fails repeatedly	route to retry topic then DLT, inspect payload
 3. Duplicate event	repeated business action	add idempotency keys and dedupe logic
 4. Broker unavailable	publish failures or ISR shrink	rely on replication, alert quickly, avoid unsafe acks
 5. Schema incompatibility	deserialization failures	enforce contract versioning and compatibility checks
 6. Rebalance storm	consumers stop/ start often	stabilize group membership and reduce long pauses



Design to recover fast, not just to avoid failure.

Kafka **Production Patterns**

Cheat Sheet Series for All Levels

Architect Decision Matrix

Decision	Choose this when	Watch out
 Increase partitions	Throughput and parallelism are the bottleneck	 Higher cost and operational complexity
 Use key-based ordering	Entity-level order matters	 Hot partitions
 Use retry topics + DLT	Transient failures are common	 More routing complexity
 Use transactional outbox	DB update and event publish must stay consistent	 Extra moving parts
 Use log compaction	Latest state per key matters	 Not a full audit history pattern by itself
 Use Kafka vs sync REST	Kafka for decoupled async event flows	 When you need immediate request/response semantics



Architects trade simplicity for reliability only when the business case justifies it.



Kafka Production Patterns

Checklist & Summary



Production Checklist

- ☐ 1. Event contract is versioned
- ☐ 2. Message key is chosen intentionally
- ☐ 3. Topic retention matches replay needs
- ☐ 4. Retry and DLT strategy is defined
- ☐ 5. Consumer processing is idempotent
- ☐ 6. Ordering assumptions are documented
- ☐ 7. Lag, errors and DLT are monitored
- ☐ 8. Load and replay are tested
- ☐ 9. Ownership and runbooks are clear
- ☐ 10. Rollback and recovery plan exists



Remember



Design for failure



Choose keys carefully



Control duplicates



Make replay safe



Observe everything



Save this cheat sheet before your next
Kafka production review.